



Buy now, pay later. Trust no one.

Hire-purchase across two chains. The asset is escrowed on Creditcoin, the installments are paid in stablecoin on Ethereum, and an installment counts only once the Attestcoin Protocol has proven its Ethereum transaction to the escrow.

Someone always holds the switch.

Buying in installments assumes a middleman: a bank, a credit provider, or in off-grid solar, an operator who can switch your panel off remotely when you stop paying. Remote lockout is the standard enforcement tool of PAYGo solar, and CGAP documents what it costs the people it is used on.

They decide

Approval, deposit and repossession are one company's call, reviewable by nobody.

They can cut you off

The lockout is remote and instant, including on a productive asset you need in order to pay.

You get nothing back

The payment record stays theirs. It never becomes credit you can carry elsewhere.

They removed the middleman and added three worse ones.

The 2022 NFT lending wave died of exactly this. PayGo has none of the three, by construction rather than by policy.

Floor-price oracles

A price feed that can be manipulated, and must be kept online forever.

Withdrawable pools

Lender capital that leaves at the worst moment, taking the credit line with it.

Off-chain operations

A team that has to keep running for the contract to keep working.

Default is the resting state. A proof is what saves you.

An inclusion proof is additive. It can establish that something happened, never that something did not. There is no proof of non-inclusion, and no amount of engineering produces one. Every design that treats the Attestcoin Protocol as an oracle to feed hits that wall. PayGo inverts it: an overdue order can be asserted in default by **anyone**, and a proof of an on-time payment cures it.

Two chains, one proof.

Verification and business logic happen in the same Creditcoin transaction. No intermediate oracle state, no privileged caller anywhere in the path.

ETHEREUM

PayGoRouter is stateless and has no admin.

The buyer pays an installment in ERC-20, in one transaction.

It emits **InstallmentPaid**, a dedicated event carrying every field the escrow will need.

ATTESTCOIN
INCLUSION PROOF

SUBMITTED
BY ANYONE

CREDITCOIN

PayGoEscrow holds the asset and the schedule.

It verifies the proof at the precompile, runs five checks, marks the installment paid.

Last one proven, the buyer pulls the asset. Silence past grace, anyone asserts default.

Every capability, running on testnet.

PROTOCOL CAPABILITY	WHERE PAYGO USES IT
verifyAndEmit, BlockProver 0x...0FD2	Before any state change, over the whole batch
Batch verification, 10 queries, one continuity proof	settle and settleCustody take 1 to 10 proofs
Attested source height, ChainInfo 0x...0fD3	The protocol's clock: what makes declareDefault permissionless and oracle-free
EvmV1Decoder, receipt and log decoding	Reading the proven receipt, linked at deploy time
Attestcoin SDK 0.18.0	waitUntilHeightAttested and getBatchProof, in the optional relayer
Source chain key 1, Sepolia on CC3	Escrow constructor, never a user parameter

The protocol proves inclusion. It does not interpret what was included.

Five checks turn a valid proof into a valid payment. Each one exists because of a property of the precompile we verified empirically first, not because it looked prudent.

- 1 **Nullifier first**, keyed on chain, height and transaction index. The precompile has no replay protection: the same proof verifies twice.
- 2 **Verify the batch**. `verifyAndEmit` reverts on an invalid proof and never returns false, so a boolean failure branch is dead code.
- 3 **`receiptStatus == 1`**. The precompile does not check it, and a reverted payment is perfectly provable.
- 4 **Pin the emitter** to the known Router and the topic to this escrow. Log lookup filters on the event signature alone.
- 5 **Business validity**, applied as a filter rather than an assertion, so a poison log cannot make a victim's payment unseizable.

Not estimates. Transactions.

-55%

gas per installment when four proofs share one continuity proof

159 912

gas per installment in that batch, against 357 476 alone

59

tests passing, plus five scanners gating every push

Batching works because every deadline is quantised onto the 1000-block grid of the protocol's continuity checkpoints, so payments from **unrelated orders** land in the same provable window and settle together. Whoever submits pays less than they would one by one. That is the incentive, and it is why no keeper has to be paid.

Three things that fall out of the same primitive.

A passport made of proofs

Every settled installment is written to an ERC-5192 soulbound record, and the escrow reads it back to size the next deposit: 40 % for a newcomer, 15 % after four honored installments. Not a score, a record of facts. Creditcoin's founding mission, rebuilt out of proofs.

Sign once, close the laptop

The buyer pre-signs the remaining installments as EIP-3009 authorizations with staggered windows. Anyone may submit them when due. The nonce binds the payee, so consent commits to where the money goes.

Proof-of-Custody

A chip signs at listing and again at delivery, through the same proof pipeline. A match releases the seller's bond. A mismatch pays nobody and burns it, because anyone can generate a keypair and paying the buyer would put a bounty on lying.

Testnet only, and it runs.

CONTRACT	CHAIN
PayGoEscrow	Creditcoin CC3
CreditPassport	Creditcoin CC3
SellerPassport	Creditcoin CC3
PayGoRouter	Sepolia
CustodyRouter	Sepolia

Addresses, transactions and a live order: **paygo.fyra.fun** and the repository README.

What was done instead of an audit

No third-party audit. An internal pre-deployment review, two automated multi-agent reviews whose findings were reproduced with executed proofs of concept, a web client review, and static analysis. Every finding ships with a regression test.

Stated plainly

Two limits are accepted by design: passport collusion between cooperating wallets, and self-forged custody on a sham order. Both are written down in the audit document rather than papered over.

No oracle. No keeper. No liquidator. No pool. **No admin key.**

The relayer is a convenience, not a trust assumption: `settle`, `declareDefault` and `finalizeDefault` are all permissionless, and a buyer who does not trust it submits the identical proof themselves. Allowlists and windows are fixed at construction. Changing a parameter means deploying again.